

网络基础与 Go 基础 | 面试常考 24 题（播客朗读版）

生成日期：2026-02-06

使用方式（推荐 25-35 分钟一遍）：

1) 每题先只背“**一句话答案**”；2) 再补“**追问要点**”；3) 睡前用播客听一遍，第二天随机抽 6-8 题复述。

覆盖范围：HTTP/HTTPS/TCP/DNS/缓存/负载均衡/排障 + Go
并发/内存/GC/GMP/slice /channel/context/interface。

Part A | 网络基础（12 题）

Q1. OSI 七层模型和 TCP/IP 四层模型怎么对应？

一句话答案：一句话：OSI 是理论分层，TCP/IP 是工程落地分层；面试按“应用-传输-网络-链路”讲最稳。

追问要点：追问要点：TCP 在传输层、IP 在网络层；链路层负责封装/MAC；应用层举例 HTTP/DNS。

Q2. 从浏览器输入 URL 到页面渲染，关键链路有哪些？

一句话答案：一句话：DNS 解析 → TCP 三次握手 → (HTTPS 再 TLS 握手) → HTTP 请求/响应 → 浏览器解析渲染。

追问要点：追问要点：缓存命中 (DNS/HTTP)；连接复用 (Keep-Alive/HTTP2)；性能瓶颈用抓包定位。

Q3. HTTP 报文结构是什么？请求与响应分别包含什么？

一句话答案：一句话：请求=请求行+请求头+空行+请求体；响应=状态行+响应头+空行+响应体。

追问要点：追问要点：Content-Type/Length；Host；Connection；Range/Accept-Ranges（断点续传）。

Q4. GET 和 POST 的区别（语义/幂等/缓存）？

一句话答案：一句话：GET 获取资源（安全、幂等、可缓存），POST 提交处理（非幂等，通常不缓存）。

追问要点：追问要点：参数位置 (URL vs body) 不代表本质；幂等与重试策略；接口设计要符合语义。

Q5. 常见状态码怎么记？502 和 504 有什么区别？

一句话答案：一句话：2xx 成功、3xx 重定向、4xx 客户端错、5xx 服务端错；502 网关收到“无效响应”，504 网关“等超时”。

追问要点：追问要点：304 缓存；401 vs 403；线上排障先看网关日志和 upstream 状态。

Q6. HTTP Keep-Alive 是什么？为什么能提升性能？

一句话答案：一句话：复用同一条 TCP 连接发多次请求，减少握手/挥手开销与慢启动成本。

追问要点：追问要点：空闲超时；连接池；在高并发下配合限流/负载均衡。

Q7. HTTP/1.1 和 HTTP/2 的核心区别？

一句话答案：一句话：HTTP/2 用二进制分帧+头部压缩+多路复用，提升并发与效率。

追问要点：追问要点：HTTP/1.1 队头阻塞；HTTP/2 仍可能被 TCP

丢包影响（同连接）；服务端推送。

Q8. HTTPS 做了什么？TLS 握手大概怎么走？

一句话答案：一句话：HTTPS=HTTP+TLS，加密+完整性校验+身份认证；握手协商会话密钥后再用对称加密传输。

追问要点：追问要点：证书链验证防中间人；三随机数思路；RSA/ECDHE；会话复用/0-RTT（了解即可）。

Q9. DNS 解析流程是什么？为什么 DNS 多用 UDP？

一句话答案：一句话：先查本地/递归解析器缓存，没命中再问根-顶级-权威；DNS 多用 UDP 因为轻量低延迟。

追问要点：追问要点：53 端口；超时重试；区域传送可能用 TCP；hosts 文件优先级。

Q10. TCP 三次握手与四次挥手：每一步在干什么？

一句话答案：一句话：握手是确认“双方收发能力”并同步序号；挥手是双方分别关闭发送方向。

追问要点：追问要点：SYN/ACK/FIN；半关闭；为什么挥手通常 4 次；RST 场景。

Q11. TIME_WAIT 为什么存在？怎么缓解 TIME_WAIT 过多？

一句话答案：一句话：TIME_WAIT

确保旧连接的报文完全消失，并保证被动关闭方能正确重传最后 ACK。

追问要点：追问要点：客户端多

TIME_WAIT；端口复用（SO_REUSEADDR/PORT）、长连接、连接池；别乱调内核参数。

Q12. 网页很慢“转圈圈”，你怎么定位是 DNS/TCP/HTTP 还是服务端？

一句话答案：一句话：分层排查：先看是否能解析，再看握手是否成功，再看响应码与首字节时间，最后看链路丢包/后端耗时。

追问要点：追问要点：抓包/Wireshark；网关/反向代理日志；区分客户端网络与服务端瓶颈。

Part B | Go 语言基础（12 题）

Q1. make 和 new 的区别？

一句话答案：一句话：new 分配零值内存并返回指针；make 只用于 slice/map/channel，分配并完成内部初始化，返回对象本身。

追问要点：追问要点：map/channel 直接 new 不能用；slice 需要 len/cap；理解“引用类型的内部结构”。

Q2. 数组和切片的区别？切片底层是什么？

一句话答案：一句话：数组长度是类型的一部分、值传递；切片是三元组（指针/len/cap）的视图，底层仍是数组。

追问要点：追问要点：切片截取共享底层数组；append 触发扩容会搬迁；传参是“值拷贝切片头”。

Q3. slice 扩容规则大概怎么说？扩容会带来什么坑？

一句话答案：一句话：容量小通常翻倍，容量大按一定比例增长；扩容会分配新数组并 copy，旧引用不再互相影响。

追问要点：追问要点：多切片共享底层数组导致“改一处全改”；预估容量减少扩容；大对象扩容影响 GC。

Q4. map 为什么遍历无序？map 并发安全吗？

一句话答案：一句话：map 遍历刻意随机化避免依赖顺序；map 非并发安全，并发读写会 panic。

追问要点：追问要点：顺序需求用 keys 排序；并发用 sync.Map 或 RWMutex；扩容搬迁导致地址不稳定。

Q5. channel 的发送/接收大致流程是什么？关闭后还能读吗？

一句话答案：一句话：优先直接与对端 goroutine 交接；其次进/出缓冲区；否则阻塞排队；关闭后仍可读完缓冲区，读到零值并返回 ok=false。

追问要点：追问要点：向已关闭 channel 发送会 panic；close 只能由发送方负责；防 goroutine 泄漏。

Q6. select 的机制与 default 有什么坑？

一句话答案：一句话：select 同时监听多个 channel，多个就绪会随机选一个；default 会让 select 变成非阻塞。

追问要点：追问要点：default 容易造成空转占 CPU；加 time.After/Timer 做超时；注意公平性与饥饿。

Q7. goroutine 和 OS 线程有什么关系？GMP 模型怎么一句话讲清？

一句话答案：一句话：G 是任务，M 是内核线程，P 是调度资源；P 绑定 M 执行 G，并通过本地队列/全局队列/偷取实现伸缩与均衡。

追问要点：追问要点：GOMAXPROCS 控制 P 数；work stealing；阻塞 syscall 会触发 M 切换。

Q8. context 在工程里怎么用？取消是怎么传播的？

一句话答案：一句话：context

用来传递取消信号、超时、链路请求级元数据；取消会沿着派生树向下广播。

追问要点：追问要点：请求入口创建 WithTimeout；goroutine select 监听 ctx.Done；避免把 context 存 struct 长期持有。

Q9. 什么是逃逸分析？怎么判断变量在栈还是堆？

一句话答案：一句话：编译器根据“是否可能在函数外被引用”等规则决定是否逃逸到堆；用 -gcflags '-m' 查看。

追问要点：追问要点：返回局部变量指针、interface 装箱、闭包引用、切片/map 扩容、大对象都易逃逸。

Q10. Go 的 GC 用的是什么算法？STW 是什么？

一句话答案：一句话：Go 用三色标记清扫，并发标记为主；STW 会暂停用户 goroutine，影响尾延迟。

追问要点：追问要点：根集合（全局/栈/寄存器）；写屏障维护三色不变性；关注 P99 延迟与 GC 触发频率。

Q11. interface 的 nil 坑：为什么 ‘var e error = (*MyErr)(nil)’ 不等于 nil？

一句话答案：一句话：interface 是 (动态类型, 动态值) 二元组；只要动态类型非空，即使动态值为 nil，接口也不等于 nil。

追问要点：追问要点：打印 %#v 看动态类型；返回 error 时避免返回“带类型的 nil”；统一返回 nil。

Q12. 如何快速定位 Go 内存/协程泄漏？你会看哪些指标？

一句话答案：一句话：用 pprof 看 heap/goroutine/profile，结合监控观察内存与 goroutine 数是否持续上升。

追问要点：追问要点：常见泄漏：阻塞 goroutine、未关闭 channel、slice 持有大数组、Timer 未 stop；用 context/退出条件兜底。

睡前 3 天冲刺建议

Day1：把 24 题各背一遍“一句话答案”；Day2：随机抽 12 题补“追问要点”；Day3：按你项目场景（RAG/Agent/高并发）把 TCP、HTTP、GMP、GC、context 这 5 题讲成系统设计。

资料来源：你提供的《计算机网络面试题 QA 整理》《Golang 面试题 QA 整理》，并做了播客口播化改写与融合。